

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 - Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	Siva Chorian	Reg. No.:	192525039
Section / Batch:	2025	Date:	04/09/2026

Code of Conduct

I, Siva Chorian (192525039) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: Siva Chorian

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

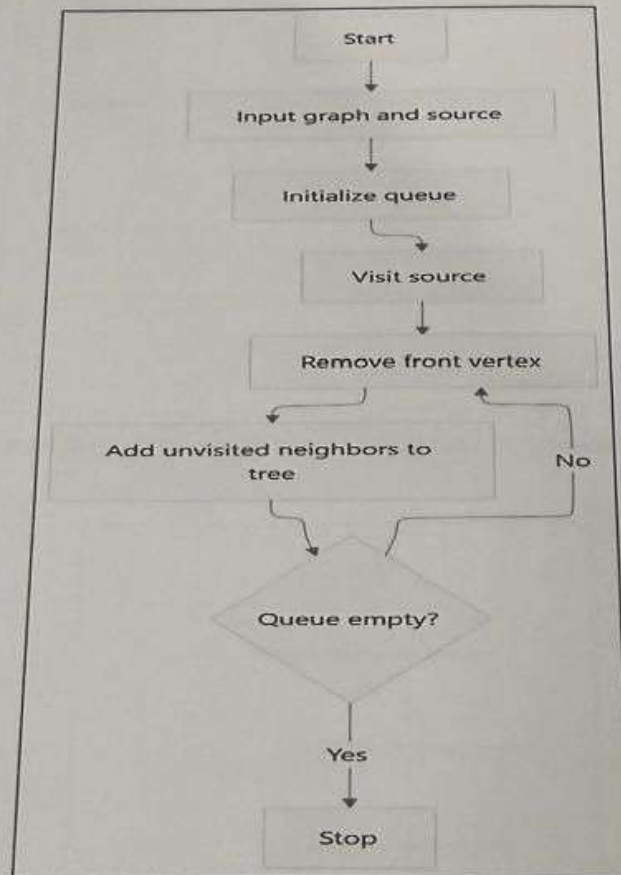
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Shortest Path Algorithm	Dijkstra's Algorithm	2	20
Shortest Path Algorithm	Unweighted Graph	3	20
Graph Traversal	BFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Pseudocode Writing Task

1. Convert the flowchart for generating a BFS tree into code.

(20 Marks)

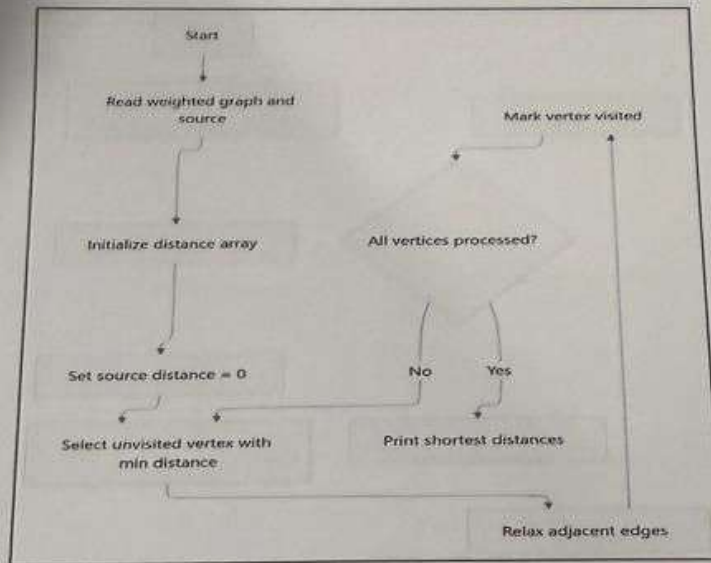
Flowchart Diagram:



2. Convert the flowchart for Dijkstra's shortest path algorithm into code.

(20 Marks)

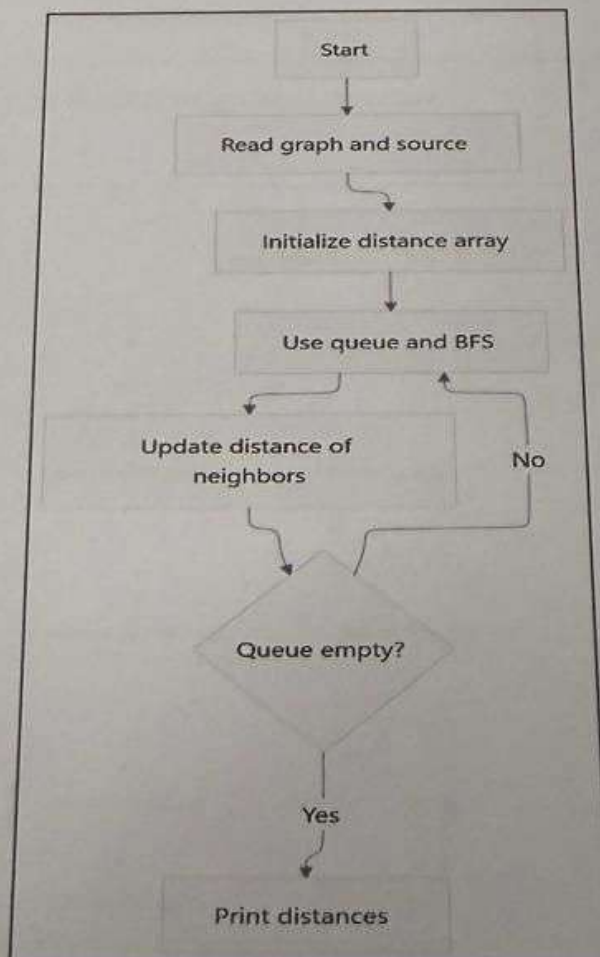
Flowchart Diagram:



3. Convert the flowchart for shortest path in an unweighted graph into code.

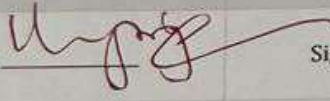
(20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

BFS - Breadth First Search

C

#include <stdio.h>

int main(){

int n, graph[20][20], visited[20] = {0};

int queue[20], front = 0, rear = 0;

int start, i, v;

printf("Enter no. of vertices:");

scanf("%d", &n);

printf("Enter adjacency matrix:\n");

for (i = 0; i < n; i++)

for (v = 0; v < n; v++)

scanf("%d", &graph[i][v]);

printf("Enter starting vertex:");

scanf("%d", &start);

queue[rear++] = start;

visited[start] = 1;

printf("BFS:");

while (front < rear) {

v = queue[front++];

printf("%d", v);

for (i = 0; i < n; i++) {

if (graph[v][i] == 1 && visited[i] == 0) {

queue[rear++] = i;

visited[i] = 1;

}

}

return 0; }

2) Dijkstra's Shortest Path

```
#include <stdio.h>
```

```
#define INF 9999
```

```
int main() {
```

```
    int n, graph[20][20], dist[20], visited[20] = {0};
```

```
    int i, j, count, u, min, source;
```

```
    printf("Enter no. of vertices: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter weighted adjacency matrix:\n");
```

```
    for (i=0; i<n; i++)
```

```
        for (j=0; j<n; j++)
```

```
            scanf("%d", &graph[i][j]);
```

```
    printf("Enter source vertex: ");
```

```
    scanf("%d", &source);
```

```
    for (count=1; count<n; count++) {
```

```
        min = INF;
```

```
        u = -1;
```

```
        for (i=0; i<n; i++) {
```

```
            if (!visited[i] && dist[i] < min) {
```

```
                min = dist[i];
```

```
                u = i;
```

```
            }
```

```
        }
```

```
        printf("Shortest distances:\n");
```

```
        for (i=0; i<n; i++)
```

```
            printf("%d -> %d = %d\n", source, i, dist[i]);
```

```
    return 0;
```

```
}
```

Shortest Path in a unweighted Graph

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, graph[20][20], dist[20], visited[20] = {0};
```

```
    int queue[20], front = 0, rear = 0;
```

```
    int source, i, v;
```

```
    printf("Enter no. of vertices:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter adjacency matrix:\n");
```

```
    for(i=0; i<n; i++)
```

```
        for(v=0; v<n; v++)
```

```
            scanf("%d", &graph[i][v]);
```

```
    printf("Enter source vertex:");
```

```
    scanf("%d", &source);
```

```
    for(i=0; i<n; i++) {
```

```
        if (graph[source][i] == 1 && !visited[i]) {
```

```
            visited[i] = 1;
```

```
            dist[i] = dist[source] + 1;
```

```
            queue[rear++] = i;
```

```
        }
```

```
    }
```

```
    printf("Shortest distances:\n");
```

```
    for(i=0; i<n; i++)
```

```
        printf("%d -> %d = %d\n", source, i, dist[i]);
```

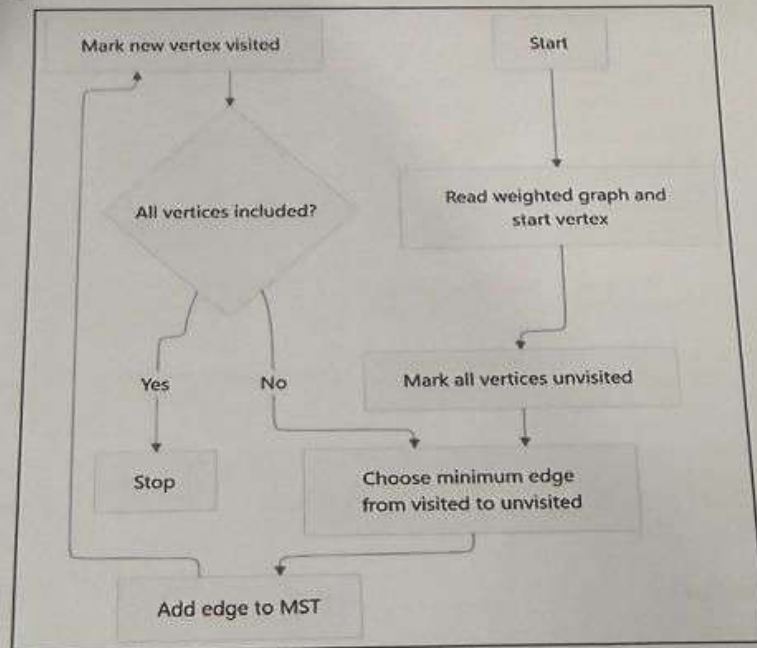
```
    return 0;
```

```
}
```


4. Convert the flowchart for Prim's algorithm into code.

(20 Marks)

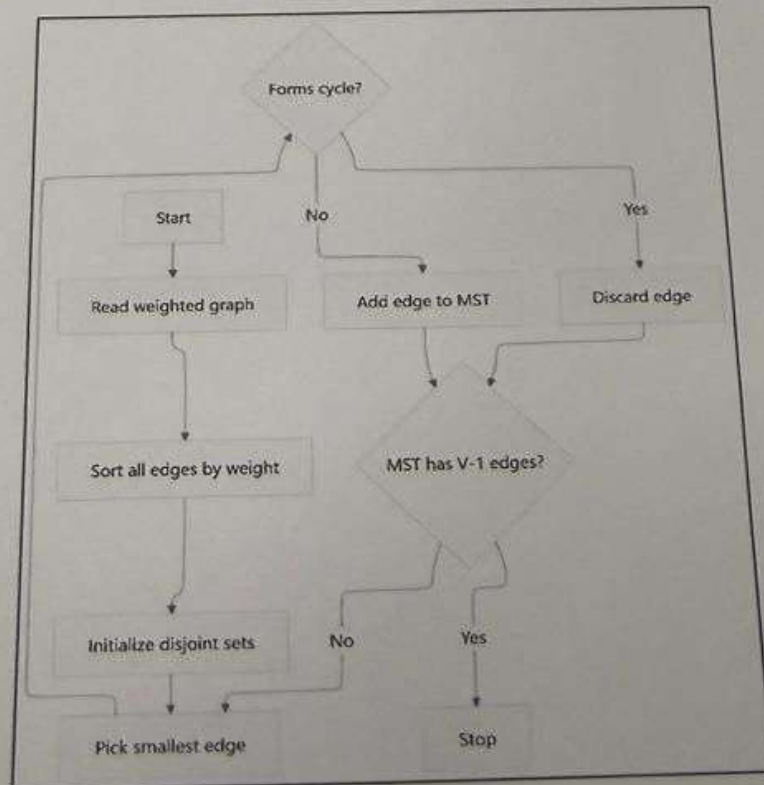
Flowchart Diagram:



5. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

Flowchart Diagram:



4) Prim's Algorithm

C

```
#include <stdio.h>
```

```
#define INF 9999
```

```
int main() {
```

```
    int n, graph[20][20], Selected[20] = {0};
```

```
    int i, j, edges = 0, x, y, min, total = 0;
```

```
    printf("Enter no. of vertices: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter weighted adjacency matrix:\n");
```

```
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < n; j++)
```

```
            scanf("%d", &graph[i][j]);
```

```
    Selected[0] = 1;
```

```
    printf("Edges in MST:\n");
```

```
    while (edges < n - 1) {
```

```
        min = INF;
```

```
        x = y = -1;
```

```
        printf("%d - %d : %d\n", x, y, min);
```

```
        total += min;
```

```
        Selected[y] = 1;
```

```
        edges++;
```

```
    }
```

```
    printf("Minimum cost = %d\n", total);
```

```
    return 0;
```

```
}
```

```
#include <stdio.h>
```

```
struct Edge {
```

```
    int u, v, w;
```

```
};
```

```
int Parent[20];
```

```
int find(int x) {
```

```
    while (Parent[x] != x)
```

```
        x = Parent[x];
```

```
    return x;
```

```
}
```

```
void unionset(int a, int b) {
```

```
    Parent[find(a)] = find(b);
```

```
}
```

```
int main() {
```

```
    struct Edge e[50], temp;
```

```
    int n, m, i, j, count = 0, total = 0;
```

```
    printf("Enter no. of vertices: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter no. of edges: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter edges (u v weight):\n");
```

```
    for (i = 0; i < m; i++)
```

```
        scanf("%d %d %d", &e[i].u, &e[i].v, &e[i].w);
```



```
for (i=0; i<n; i++)
```

```
parent[i] = i;
```

```
for (i=0; i<m-1; i++) {
```

```
for (j=i+1; j<m; j++) {
```

```
if (e[i].w > e[j].w) {
```

```
temp = e[i];
```

```
e[i] = e[j];
```

```
e[j] = temp;
```

```
}
```

```
}
```

```
}
```

```
printf("Edges in MST:\n");
```

```
for (i=0; i<m && count < n-1; i++) {
```

```
int a = find(e[i].u);
```

```
int b = find(e[i].v);
```

```
if (a != b) {
```

```
printf("%d - %d : %d\n",
```

```
e[i].u, e[i].v, e[i].w);
```

```
total += e[i].w;
```

```
unionset(a, b);
```

```
count++;
```

```
}
```

```
}
```

```
printf("Minimum cost = %d\n", total);
```

```
return 0;
```

```
}
```

Ugly